

UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Projet PF22

RAPPORT DE PROJET

**Réduction des Coûts d'Apprentissage
par Approche Multi-Fidélité**

Ce projet explore la possibilité de réduire drastiquement les coûts de constitution d'un jeu de données en combinant une faible proportion d'images **Haute Fidélité** avec une grande majorité d'images **Basse Fidélité** acquises à moindre coût.

Auteur : Ivann Vasic

Période : Mars – Juin 2026

Cadre : Projet PF22 — UTBM

Résumé

L'entraînement de modèles d'apprentissage profond performants nécessite traditionnellement de vastes ensembles de données de haute qualité, dont l'acquisition et l'annotation s'avèrent souvent coûteuses en temps et en ressources. Ce projet explore une approche alternative : **l'apprentissage multi-fidélité**.

L'objectif est de démontrer qu'il est possible de réduire drastiquement les coûts de constitution d'un jeu de données en combinant une faible proportion d'images de **Haute Fidélité (HF)** avec une grande majorité d'images dégradées acquises à moindre coût (**Basse Fidélité - BF**).

Mots-clés : Apprentissage profond · Multi-fidélité · CNN · ResNet-18 · Dégradation d'image · Optimisation des coûts · Domain Shift · Animals-10

Table des matières

1	Notions Préliminaires	5
1.1	Les réseaux de neurones artificiels	5
1.2	Paramètres et poids du réseau	5
1.3	La boucle d'entraînement	6
1.3.1	La fonction de perte	6
1.3.2	La descente de gradient et l'optimiseur	6
1.4	Hyperparamètres clés	7
1.5	Sous-apprentissage, sur-apprentissage et généralisation	7
1.6	Les différentes dimensions du coût en apprentissage profond	8
2	Fondation et construction de l'environnement Multi-Fidélité	9
2.1	Environnement de travail et architecture hybride	9
2.2	Sélection du jeu de données	9
2.3	Découpage et répartition des données	10
2.4	Pipeline de dégradation de la qualité	11
2.5	Définition de la métrique de coût	11
2.6	Sélection et justification de l'architecture neuronale	12
2.6.1	Mécanique interne de ResNet-18	12
2.6.2	Justification des hyperparamètres et de la fonction de coût	12
2.6.3	Prétraitement et optimisations matérielles	13
3	Construction des Modèles de Référence	14
3.1	Stratégie d'évaluation croisée face au décalage de domaine	14
3.2	Définition des modèles de référence	14
3.3	Stratégie d'implémentation logicielle	15
3.4	Analyse des résultats	15
4	Stratégies Multi-Fidélité	18

4.1	Stratégie 1 – Transfer Learning Séquentiel (BF $\rightarrow$ HF)	18
4.1.1	Protocole	18
4.1.2	Résultats quantitatifs obtenus	19
4.1.3	Comparaison avec les baselines du Chapitre 3	20
4.1.4	Analyse : ce qui a marché, ce qui a moins marché, et enseignements	21
4.1.5	Améliorations possibles de la méthode (Stratégie 1)	21
4.2	Stratégie 2 – Co-training par Batch avec Pondération (Reweighting)	22
4.2.1	Protocole	22
4.2.2	Résultats quantitatifs obtenus	23
4.2.3	Comparaison avec les baselines et la Stratégie 1	25
4.2.4	Analyse : ce qui a marché, ce qui a moins marché, et enseignements	26
4.2.5	Améliorations possibles de la méthode (Stratégie 2)	27
4.3	Stratégie 3 – Teacher-Student inspirée de Noisy Student	27
4.3.1	Protocole	28
4.3.2	Résultats quantitatifs obtenus	28
4.3.3	Comparaison avec les autres stratégies	29
4.3.4	Analyse : pourquoi la stratégie 3 échoue dans ce contexte	30
4.3.5	Améliorations possibles de la méthode (Stratégie 3)	30

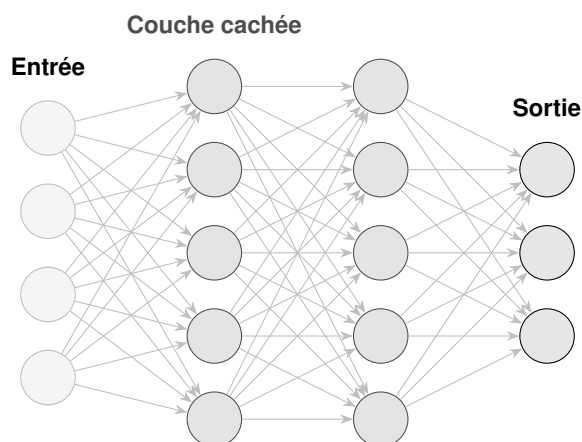
CHAPITRE 1

Notions Préliminaires

Ce chapitre présente les concepts fondamentaux nécessaires à la compréhension du projet. Le lecteur familier avec l'apprentissage profond peut passer directement au Chapitre 2.

1.1 Les réseaux de neurones artificiels

Un réseau de neurones artificiel est un système de calcul inspiré du cerveau biologique. Il est constitué d'unités élémentaires, les **neurones**, organisées en **couches** successives. Chaque neurone reçoit des signaux numériques en entrée, les combine linéairement, puis applique une **fonction d'activation** non-linéaire avant de transmettre le résultat à la couche suivante.



L'empilement de plusieurs couches cachées constitue un réseau dit **profond** (*deep network*). La profondeur permet au réseau d'apprendre des représentations de plus en plus abstraites : les premières couches détectent des motifs simples (contours, textures), tandis que les couches profondes reconnaissent des concepts complexes (forme d'un animal, expression d'un visage).

1.2 Paramètres et poids du réseau

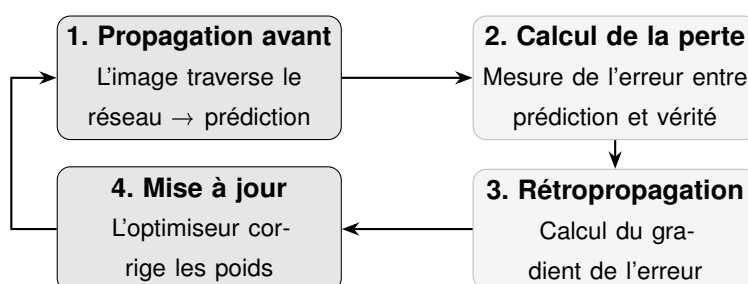
Les connexions entre neurones sont régies par des valeurs numériques appelées **poids** (w) et **biais** (b). Ce sont les **paramètres apprenables** du réseau : ils sont initialisés aléatoirement, puis ajustés itérativement au cours de l'entraînement.

Paramètre	Rôle	Interprétation
Poids w_{ij}	Pondère la connexion entre le neurone i et j	Fort poids $\Rightarrow$ le signal du neurone i influence fortement j
Biais b_j	Décale le seuil d'activation du neurone j	Permet au neurone de s'activer même en absence de signal fort

Un réseau comme ResNet-18 possède environ **11 millions de paramètres**. La qualité de l'apprentissage dépend de la capacité à trouver les valeurs optimales de l'ensemble de ces paramètres.

1.3 La boucle d'entraînement

L'entraînement d'un réseau de neurones est un processus itératif qui suit quatre étapes fondamentales, répétées pour chaque lot (*batch*) d'images :



Une **époque** (*epoch*) correspond au passage complet de l'intégralité du jeu de données d'entraînement à travers cette boucle. L'entraînement se déroule sur plusieurs dizaines d'époques jusqu'à convergence.

1.3.1 La fonction de perte

La **fonction de perte** (*loss function*) quantifie l'écart entre la prédiction du réseau et la vérité terrain. Plus la perte est faible, plus le modèle est précis. Pour la classification multi-classes, la **Entropie Croisée** (*Cross-Entropy Loss*) est le standard :

$$\mathcal{L} = - \sum_{c=1}^C y_c \cdot \log(\hat{p}_c)$$

où C est le nombre de classes, y_c la vraie étiquette (0 ou 1), et $\hat{p}_c$ la probabilité prédite par le réseau pour la classe c .

1.3.2 La descente de gradient et l'optimiseur

La **rétropropagation** calcule, pour chaque paramètre du réseau, le gradient de la perte : une mesure indiquant dans quelle direction et de quelle amplitude modifier ce paramètre pour réduire l'erreur.

L'**optimiseur** utilise ensuite ces gradients pour mettre à jour les poids :

$$w \leftarrow w - \eta \cdot \nabla_w \mathcal{L}$$

où η est le **taux d'apprentissage** (*learning rate*), un hyperparamètre critique qui contrôle la taille du pas de correction. Un η trop grand risque de faire diverger l'apprentissage ; trop petit, la convergence devient extrêmement lente.

1.4 Hyperparamètres clés

Contrairement aux poids, les **hyperparamètres** ne sont pas appris automatiquement : ils doivent être fixés manuellement avant l'entraînement et conditionnent fortement la qualité du résultat final.

Hyperparamètre	Définition	Impact
Taux d'apprentissage η	Amplitude du pas de correction	Trop grand : divergence. Trop petit : apprentissage lent ou bloqué.
Taille de lot (<i>batch size</i>)	Nombre d'images traitées simultanément	Influence la stabilité du gradient et l'utilisation mémoire du GPU.
Nombre d'époques	Nombre de passages complets sur les données	Trop faible : sous-apprentissage. Trop élevé : mémorisation (<i>overfitting</i>).
Architecture	Nombre et type de couches	Détermine la capacité d'abstraction et la vitesse d'inférence.

1.5 Sous-apprentissage, sur-apprentissage et généralisation

L'objectif de l'entraînement n'est pas de mémoriser parfaitement les données d'entraînement, mais de **généraliser** : être capable de classer correctement des images jamais vues. Deux pathologies opposées peuvent survenir :

Phénomène	Cause	Symptôme
Sous-apprentissage (<i>Underfitting</i>)	Réseau trop simple ou entraînement trop court	Mauvaises performances aussi bien en entraînement qu'en test.
Sur-apprentissage (<i>Overfitting</i>)	Réseau trop complexe ou trop peu de données	Excellentes performances en entraînement, chute sévère en test.

Le découpage systématique du jeu de données en un ensemble d'**entraînement** et un ensemble de **test** (jamais utilisé pendant l'apprentissage) permet de détecter ces phénomènes et de mesurer objectivement la généralisation du modèle.

1.6 Les différentes dimensions du coût en apprentissage profond

En apprentissage profond industriel, le terme « coût » recouvre trois réalités distinctes qu'il convient de ne pas confondre :

#	Type de coût	Nature	Exemples concrets
1	Coût de la donnée	Acquisition et annotation du jeu de données	Photographe professionnel, expert annotateur humain, équipement haute résolution.
2	Coût de calcul	Temps GPU nécessaire à l'entraînement	Location d'instances cloud, consommation électrique, durée d'entraînement.
3	Coût de déploiement	Inférence en production	Taille du modèle, latence de prédiction, mémoire requise.

Ce projet se concentre prioritairement sur le **Coût de la donnée** (type 1), qui représente le verrou économique principal dans la plupart des applications industrielles réelles. La métrique de **Crédit d'Annotation (CA)**, définie au Chapitre 1, permet de quantifier ce coût de manière rigoureuse et indépendante du matériel utilisé.

Résumé des notions clés

Un réseau de neurones apprend en ajustant itérativement ses **poids** pour minimiser une **fonction de perte**, via la **rétropropagation du gradient**. La qualité de cet apprentissage dépend des **hyperparamètres**, du **volume** et de la **qualité** des données d'entraînement. L'enjeu central de ce projet est de démontrer qu'il est possible de réduire le **coût d'acquisition des données** sans sacrifier les performances de généralisation.

CHAPITRE 2

Fondation et construction de l'environnement Multi-Fidélité

Les choix et la mise en place de l'environnement de travail ont été guidés par des contraintes matérielles et par la nécessité de disposer de données adaptées à la simulation d'une approche multi-fidélité.

2.1 Environnement de travail et architecture hybride

Une **architecture hybride déportée** a été conçue, alliant faible coût et haute efficacité, en cohérence directe avec la thématique du projet.

Outil	Rôle	Détail
VS Code	IDE local	Écriture du code, gestion des terminaux et suivi de projet
Google Colab	Calcul distant	GPU NVIDIA T4 cloud, connecté directement à VS Code via extension sécurisée
Google Drive	Stockage persistant	Monté à chaque session ; héberge le dataset et sauvegarde les <i>checkpoints</i> par époque
GitHub	Versioning	Contrôle de version et sauvegarde du code source expérimental

Un script de test d'intégration a été développé pour valider cette synergie. Il a mis en évidence un **goulot d'étranglement réseau (I/O Bottleneck)** lors de la lecture de milliers de fichiers via le protocole FUSE de Drive, résolu par l'extraction locale d'archives ZIP directement sur le SSD de l'instance Colab.

2.2 Sélection du jeu de données

Le jeu de données doit être suffisamment difficile pour que la dégradation visuelle impacte mesurément les performances du réseau. Cinq candidats ont été évalués :

Dataset	Images	Avantages	Inconvénients
Imagenette	13 000	Standard académique, itérations rapides	Trop « facile » pour ResNet-18 ($\approx 95\%$ sans dégradation)
STL-10	105 000	Grand jeu non étiqueté (100k images)	Résolution native 96×96 trop faible
Oxford Pets	7 400	Classification fine-grained, très sensible aux dégradations	Volume insuffisant, nécessite un pré-entraînement lourd
Animals-10	26 179	Haute résolution, volume idéal, proche des conditions réelles	Bruit de labels et filigranes (peuvent être corrigés)
Imagewoof	12 954	10 races de chiens visuellement très proches, haute résolution	Volume environ trois fois inférieur à Animals-10, risque de sur-apprentissage

Choix retenu : Animals-10 et Imagewoof (double validation)

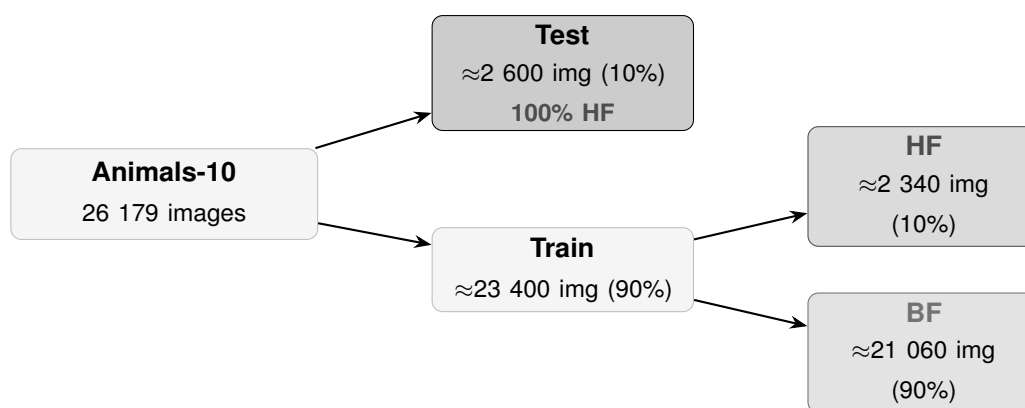
Afin de garantir que les conclusions tirées sur les stratégies multi-fidélité ne sont pas un artefact propre à un dataset particulier, **deux jeux de données ont été retenus** et le protocole expérimental complet est exécuté sur chacun :

- **Animals-10** (26 179 images, 10 classes d'animaux) — dataset principal d'expérimentation. Son volume permet un entraînement *from scratch* stable avec ResNet-18, sa haute résolution offre une large plage dynamique pour le pipeline de dégradation, et son caractère communautaire le rapproche des conditions industrielles réelles.
- **Imagewoof** (12 954 images, 10 races de chiens) — dataset de **validation croisée**. Sous-ensemble d'ImageNet plus difficile (classification *fine-grained* de classes visuellement très proches) et nettement plus petit en volume, il met les stratégies sous pression. Une stratégie qui reste la meilleure sur les deux datasets peut alors être considérée comme **robuste** et non liée aux spécificités d'un seul jeu de données.

Les deux datasets suivent strictement le même pipeline (répartition test 10 % / train 90 %, train découpé en 10 % HF + 90 % BF, même dégradation, ResNet-18 *from scratch*, 20 époques, AMP FP16).

2.3 Découpage et répartition des données

Afin d'étudier l'impact de la qualité des données sur les performances d'apprentissage, un environnement expérimental a été construit : découpage asymétrique du dataset, pipeline de dégradation et métrique de coût. Le schéma ci-dessous illustre le découpage du jeu Animals-10.

**Principe clé**

Le jeu de **test est exclusivement HF** pour toutes les expériences : l'évaluation doit mesurer la capacité de généralisation sur des données de haute qualité, quel que soit le domaine utilisé à l'entraînement.

2.4 Pipeline de dégradation de la qualité

La génération des images Basse Fidélité applique deux transformations consécutives **à la volée** :

#	Transformation	Description	Effet simulé
1	Sous-échantillonnage (<i>Downsampling</i>)	Réduction à 64×64 px puis redimensionnement à 224×224 px par interpolation bilinéaire	Perte d'information : image floue et pixélisée
2	Bruit Gaussien	Ajout d'un bruit aléatoire (grain) aux tenseurs de l'image	Simulation du bruit ISO des capteurs électroniques

2.5 Définition de la métrique de coût

Type d'image	Coût	Justification
Haute Fidélité (C_{HF})	10 CA	Expert humain ou équipement onéreux
Basse Fidélité (C_{BF})	1 CA	Collecte automatisée, pré-étiquetage web

Pourquoi pas le temps de calcul ?

Le temps de calcul brut ($Temps_{Epoch} \times Nombre_{Epochs}$) a été écarté en raison de l'instabilité des environnements cloud (fluctuations de charge serveur, allocation dynamique GPU, goulots I/O). Le **Crédit d'Annotation** offre un référentiel **invariant, agnostique au matériel**, centré sur la valeur intrinsèque de la donnée.

2.6 Sélection et justification de l'architecture neuronale

Pour que la comparaison entre nos différents jeux de données (HF, BF, Mixte) soit scientifiquement valide, l'architecture doit rester strictement identique. Plusieurs architectures standards de vision par ordinateur ont été évaluées :

Architecture	Paramètres	Points forts	Limites / Pertinence
ResNet-18 ✓	≈11 M	Standard académique, connexions résiduelles, robuste au bruit.	Pertinence excellente — compromis idéal.
MobileNetV2	≈3,4 M	Ultra-léger, économe en VRAM, itérations rapides.	S'effondre sur données fortement bruitées.
EfficientNet-B0	≈5,3 M	Ratio précision/paramètres optimisé mathématiquement.	Difficile à faire converger <i>from scratch</i> .
ViT-Tiny	≈5 M	Architecture Transformer, vision globale.	Sur-apprend massivement sans pré-entraînement.

Architecture retenue : ResNet-18 (Entraîné *from scratch*)

Ses ≈11 millions de paramètres lui confèrent une capacité d'abstraction suffisante pour extraire des caractéristiques à travers le bruit. Il a été délibérément choisi de ne pas utiliser de poids pré-entraînés (`weights=None`) afin de mesurer le véritable impact qualitatif de notre jeu de données Animals-10 sur l'apprentissage, sans biais cognitif préalable. La dernière couche (classifieur) a été adaptée pour générer 10 vecteurs de sortie correspondant à nos 10 classes animales.

2.6.1 Mécanique interne de ResNet-18

L'architecture *Residual Network* (ResNet) a révolutionné l'apprentissage profond en introduisant les **connexions résiduelles** (*skip connections*). Dans un réseau convolutionnel classique profond, le signal du gradient a tendance à s'estomper lors de la phase de rétropropagation (le problème de la disparition du gradient). ResNet contourne ce problème en ajoutant l'entrée d'une couche directement à sa sortie mathématique ($F(x) + x$).

Dans le contexte de données Basse Fidélité (flou et bruit gaussien), cette architecture est cruciale : si une couche convolutionnelle n'arrive pas à extraire d'information pertinente à cause du bruit, la connexion résiduelle permet à l'information globale de l'image de continuer à circuler librement à travers le réseau sans être détruite. La version à 18 couches (ResNet-18) offre le compromis parfait entre expressivité et prévention du sur-apprentissage (*overfitting*) sur notre corpus de ≈25 000 images.

2.6.2 Justification des hyperparamètres et de la fonction de coût

Pour garantir la reproductibilité et la comparabilité des *Baselines*, une configuration stricte d'hyperparamètres a été codée en dur dans notre script d'entraînement (`train_baselines.py`) :

- **Fonction de perte (Loss)** – `CrossEntropyLoss` : Il s'agit du standard mathématique pour les problèmes de classification multi-classes. Cette fonction combine une activation Softmax (pour transformer les sorties brutes du modèle en probabilités de 0 à 1) et une perte log-vraisemblance négative (NLL). Elle pénalise fortement le modèle s'il est très confiant sur une mauvaise prédiction, ce qui est idéal pour forcer le réseau à douter sur des images floues.
- **Optimiseur** – `Adam` : Contrairement à une descente de gradient stochastique classique (SGD), Adam adapte dynamiquement le taux d'apprentissage pour chaque paramètre individuel en utilisant les moments du premier et second ordre des gradients. Cela permet une convergence beaucoup plus rapide et stable, particulièrement utile lors d'un entraînement *from scratch*.
- **Taux d'apprentissage (Learning Rate)** : Fixé à $lr = 0.001$. C'est la valeur par défaut recommandée pour l'optimiseur Adam, offrant un pas de descente suffisamment grand pour échapper aux minima locaux lors des premières époques, tout en évitant la divergence.
- **Taille de lot (Batch Size)** : Fixée à 64. Cette valeur permet d'optimiser l'utilisation de la mémoire vidéo (VRAM) du GPU T4, tout en garantissant un gradient stochastique suffisamment lissé pour ne pas rendre l'apprentissage chaotique face au bruit des images BF.
- **Nombre d'époques** : Fixé à 20. Des tests préliminaires ont montré qu'au-delà de 20 époques, la courbe d'apprentissage sur le jeu d'entraînement atteint un plateau asymptotique, et le risque de mémorisation du bruit (*overfitting*) augmente considérablement.

2.6.3 Prétraitement et optimisations matérielles

Afin d'exploiter pleinement l'architecture hybride (VS Code / Google Colab), deux optimisations logicielles majeures ont été intégrées à l'implémentation :

1. **Normalisation des tenseurs** : Avant d'entrer dans le réseau, toutes les images (redimensionnées en 224×224 pixels) subissent une normalisation spatiale basée sur les statistiques du jeu de données ImageNet (Moyennes : $[0.485, 0.456, 0.406]$, Écarts-types : $[0.229, 0.224, 0.225]$). Cette opération centre les données autour de zéro, accélérant la convergence de la descente de gradient.
2. **Précision Mixte Automatique (AMP)** : L'entraînement utilise les modules `torch.amp.autocast` et `GradScaler`. Cette technique exécute certaines opérations mathématiques en virgule flottante 16 bits (FP16) au lieu de 32 bits (FP32). Cela a permis de réduire drastiquement la consommation mémoire et d'accélérer le temps de calcul sur les Tensor Cores du GPU NVIDIA T4, réduisant la durée d'une époque de plusieurs minutes à une quarantaine de secondes, sans aucune perte de précision prédictive.

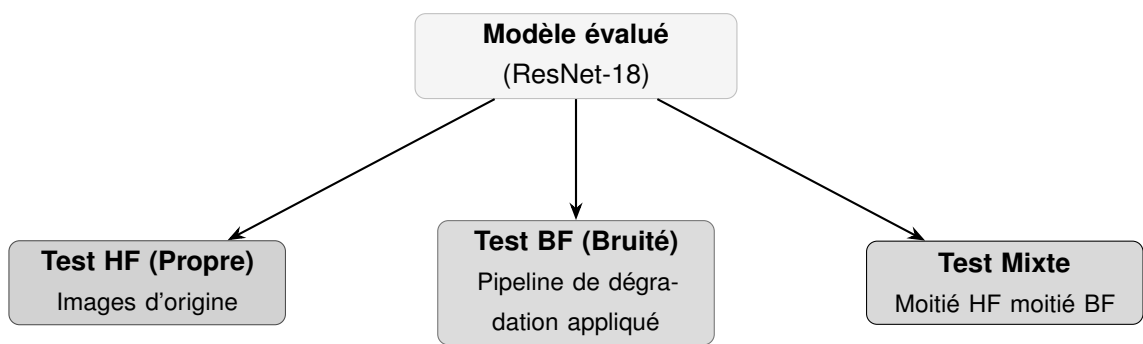
CHAPITRE 3

Construction des Modèles de Référence

Avant d’explorer des stratégies d’optimisation complexes, il est indispensable d’établir des **bornes de performances** (supérieure et inférieure) via des *baselines*.

3.1 Stratégie d’évaluation croisée face au décalage de domaine

Un modèle entraîné sur des images floues (domaine BF) peut ne pas généraliser correctement sur des images nettes (domaine HF) — c’est le **Domain Shift**. Pour quantifier ce phénomène, chaque modèle est évalué selon **trois passages distincts** :



3.2 Définition des modèles de référence

#	Modèle	Images	Coût	Hypothèse de comportement
1	Premium (Train HF uniquement)	≈2 500 HF	25 000 CA	Excellentes performances sur Test HF. Effondrement prévu sur Test BF (aucune robustesse au bruit).
2	Low-Cost (Train BF uniquement)	≈22 700 BF	22 700 CA	Très performant sur Test BF. Généralisation sur Test HF dépendra de la capacité à extrapoler des contours nets depuis des signaux flous.
3	Naïf Mixte (Borne supérieure)	≈25 200 HF+BF	47 700 CA	Meilleures performances globales (Test Mixte). Sert de borne supérieure absolue. L’objectif des phases suivantes est d’atteindre ce niveau à coût réduit.

3.3 Stratégie d'implémentation logicielle

Afin d'éliminer tout biais d'implémentation, le code a été factorisé en un **unique script d'entraînement paramétrable**. Ce script accepte un argument de mode (HF, BF, ou MIXTE), charge dynamiquement le sous-ensemble correspondant, procède à l'entraînement et exécute automatiquement la séquence des trois tests d'évaluation croisée.

3.4 Analyse des résultats

Les trois modèles de référence ont été entraînés avec succès sur 20 époques, sur chacun des deux datasets retenus. Les résultats obtenus, résumés dans le tableau ci-dessous, mettent en évidence la dynamique du compromis coût/performance.

Modèle Entraîné	Coût Total (CA)	Test HF	Test BF	Test Mixte	Temps d'entraînement
<i>Animals-10</i>					
BL 1 (HF)	470 400	56,20 %	35,16 %	45,68 %	2,84 min
BL 2 (BF)	424 260	59,07 %	63,66 %	61,36 %	27,05 min
BL 3 (Mixte)	894 660	75,90 %	65,03 %	70,47 %	29,78 min
<i>Imagewoof</i>					
BL 1 (HF)	179 800	24,87 %	19,04 %	21,95 %	0,39 min
BL 2 (BF)	162 520	43,73 %	44,52 %	44,12 %	1,00 min
BL 3 (Mixte)	342 320	47,19 %	47,24 %	47,21 %	1,36 min

L'analyse de ces métriques sur **Animals-10** (dataset principal) permet de tirer trois conclusions majeures concernant le comportement du réseau ResNet-18 face à nos données :

- **L'effondrement face au bruit (*Domain Shift*)** : La Baseline 1, entraînée exclusivement sur des données parfaites, obtient un score de 56,20 % sur le test HF. Cependant, confrontée au domaine BF (flou et bruit gaussien), sa précision s'effondre à 35,16 %. Ce résultat prouve l'incapacité d'un modèle "Premium" classique à généraliser face à des perturbations non rencontrées lors de l'apprentissage.
- **Le paradoxe de la quantité** : La Baseline 2 possède un budget inférieur à la Baseline 1 (424 260 CA contre 470 400 CA) mais la surpasse systématiquement, **y compris sur les images nettes** (59,07 %). Cela démontre qu'en apprentissage profond, l'immense diversité contenue dans $\approx 22\,700$ images floues prime sur la perfection visuelle de seulement $\approx 2\,500$ images.

- **La validation de la borne supérieure** : La Baseline 3 confirme son statut théorique. En ayant appris simultanément sur le volume massif (BF) et sur les détails discriminants (HF), elle atteint une précision de 75,9 % sur le test de haute fidélité. Toutefois, son coût d'acquisition est beaucoup plus élevé que les autres baseline.

Comparaison des 3 baselines

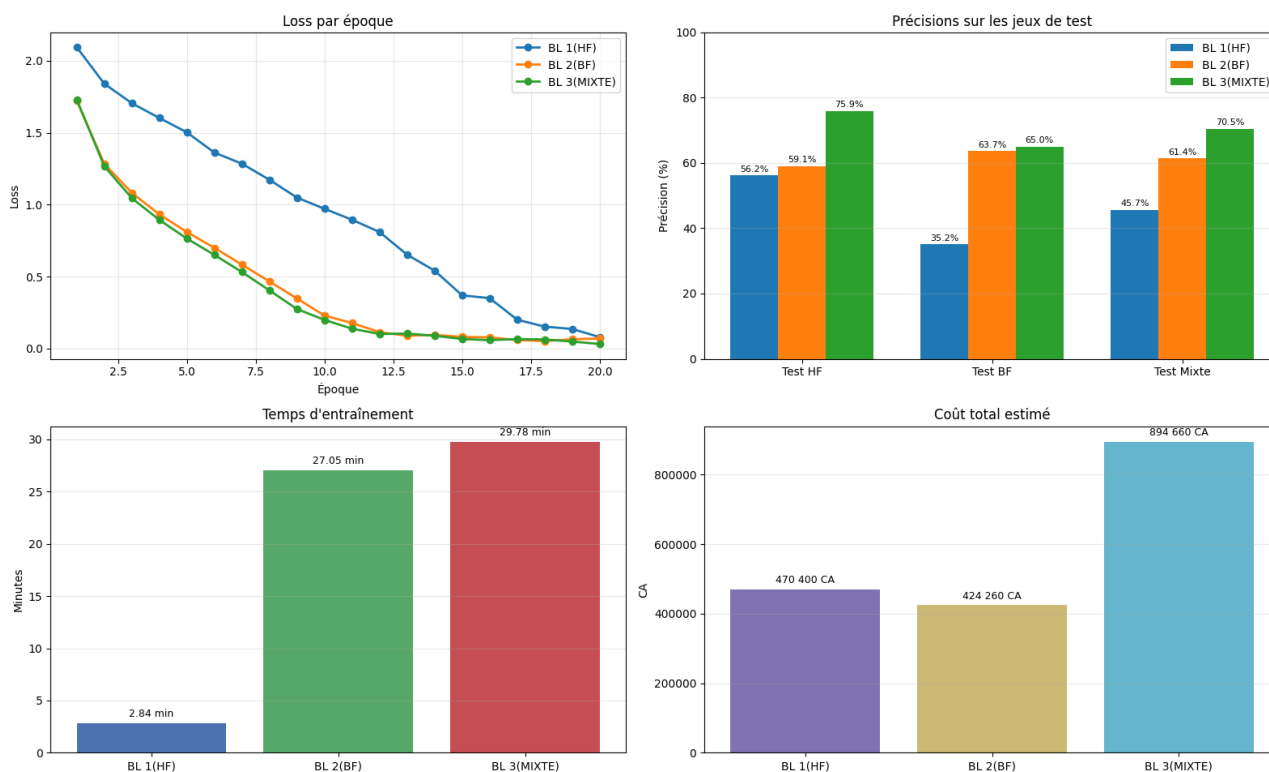


FIGURE 3.1 – Comparaison visuelle des trois baselines sur **Animals-10** (BL 1(HF), BL 2(BF), BL 3(MIXTE)) : loss par époque, précisions de test, temps d'entraînement et coût total estimé.

Vérification sur Imagewoof

Réexécutées sur Imagewoof, les trois baselines confirment les conclusions tirées sur Animals-10, malgré des valeurs absolues nettement inférieures (Imagewoof = classification *fine-grained* de races canines, $\approx 3\times$ moins d'images d'entraînement) :

- **Domain Shift confirmé** : la BL 1 passe de 24,87 % (HF) à 19,04 % (BF), soit une chute de $-5,8$ points cohérente avec celle observée sur Animals-10.
- **Paradoxe de la quantité confirmé** : la BL 2 (162 520 CA) bat la BL 1 (179 800 CA) sur les trois tests, y compris HF (43,73 % vs 24,87 %), avec un budget inférieur.
- **Borne supérieure validée** : la BL 3 atteint 47,19 % sur HF pour un coût double de BL 1.

La hiérarchie **BL 3 > BL 2 > BL 1** est strictement conservée d'un dataset à l'autre, ce qui valide la robustesse des conclusions sur le comportement des baselines indépendamment du dataset utilisé.



FIGURE 3.2 – Comparaison visuelle des trois baselines sur **Imagewoof** : précisions de test, temps d'entraînement et coût total estimé. Les valeurs absolues sont plus basses qu'Animals-10 (classification fine-grained) mais la hiérarchie $BL\ 3 > BL\ 2 > BL\ 1$ est strictement identique.

CHAPITRE 4

Stratégies Multi-Fidélité

Ce chapitre inaugure la Partie 4 et présente les premières stratégies multi-fidélité orientées **objectif HF prioritaire**. Après une approche séquentielle (Étape 1), nous analysons une approche conjointe (Étape 2) où les données HF et BF sont vues simultanément avec une pondération explicite de la perte.

4.1 Stratégie 1 – Transfer Learning Séquentiel (BF → HF)

4.1.1 Protocole

La stratégie retenue repose sur deux phases d'entraînement consécutives avec la même architecture ResNet-18 :

1. **Pré-entraînement BF** : apprentissage sur le volume principal de données dégradées pour acquérir une représentation robuste et générique des classes.
2. **Fine-tuning HF** : spécialisation sur un sous-ensemble HF plus petit, afin de récupérer les détails discriminants de haute fidélité.

Configuration expérimentale utilisée :

Paramètre	Valeur	Commentaire
Architecture	ResNet-18	Entraînée <i>from scratch</i> , tête de classification 10 classes
Optimiseur	Adam	Identique aux baselines pour conserver la comparabilité
Batch size	64	Compromis stabilité/vitesse sur GPU T4
Époques BF	10	21 213 images BF (332 batches/époque)
Époques HF	10	1 881 images HF train + 471 images HF val
Learning rates	0,001 puis 0,0003	Plus faible en fine-tuning pour stabiliser l'adaptation
Accélération	AMP (FP16)	autocast + GradScaler

4.1.2 Résultats quantitatifs obtenus

Les résultats finaux de l'Étape 1 sont résumés ci-dessous pour les deux datasets. Dans cette étude, le critère prioritaire est la performance sur **Test HF**.

Méthode	Test HF	Test BF	Test Mixte	Coût (CA)
<i>Animals-10</i>				
Stratégie 1 (BF → HF)	78,84%	57,65%	68,25%	400 230
<i>Imagewoof</i>				
Stratégie 1 (BF → HF)	53,68%	44,82%	49,25%	153 160

Lecture rapide des courbes d'entraînement (Animals-10)

La phase BF converge rapidement en *train loss* (1,694 → 0,226), alors que la validation HF fluctue fortement (39,28% → 57,75% puis 50,96%). Après transition vers HF, la précision validation HF bondit immédiatement autour de 78–79%, puis se stabilise ; la *val loss* remonte légèrement en fin de fine-tuning, signalant un début de sur-apprentissage.

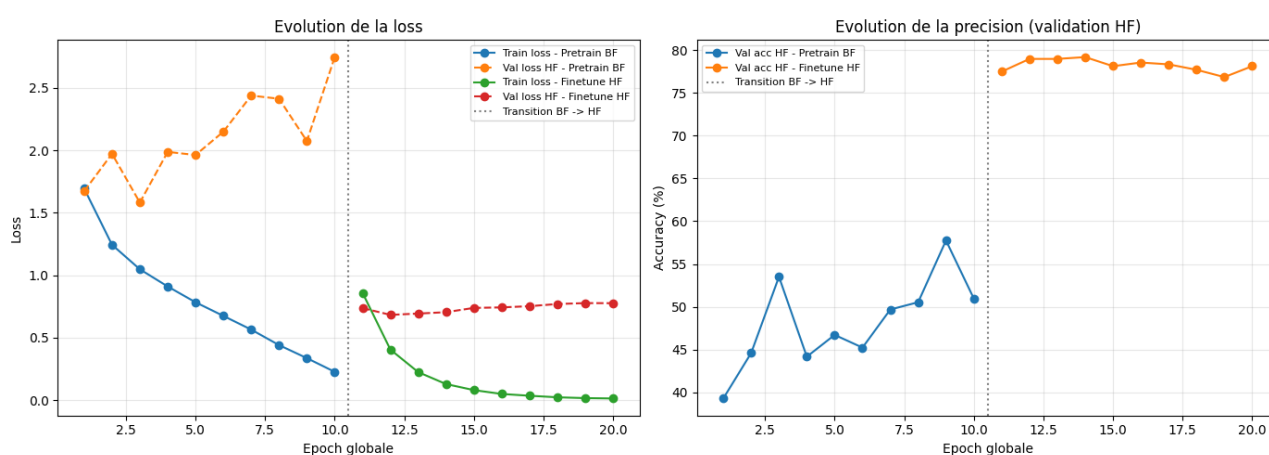


FIGURE 4.1 – Évolution de la loss et de la précision validation HF pendant le pré-entraînement BF puis le fine-tuning HF, sur **Animals-10**.

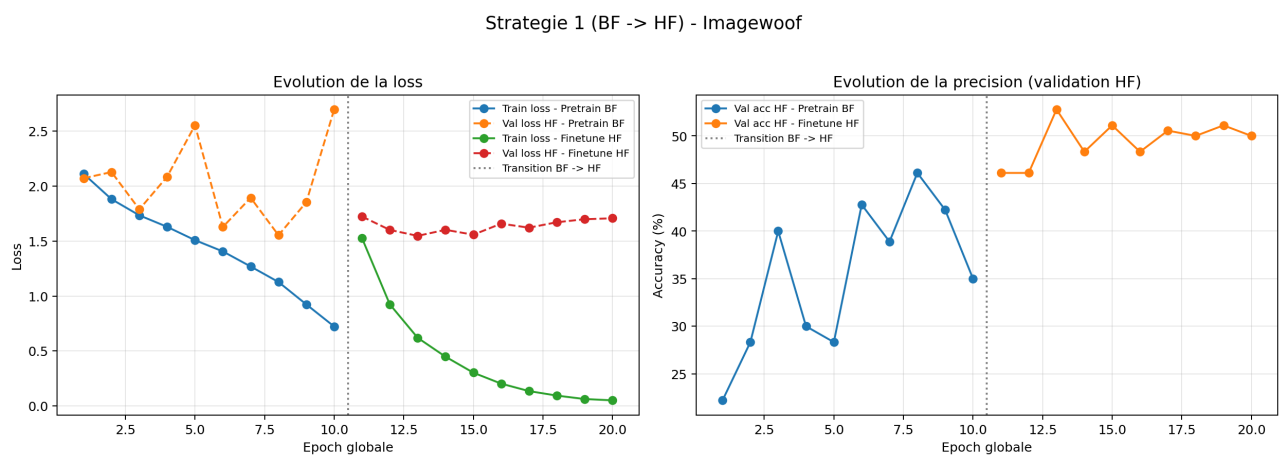


FIGURE 4.2 – Évolution de la loss et de la précision validation HF pendant le pré-entraînement BF puis le fine-tuning HF, sur **Imagewoof**.

4.1.3 Comparaison avec les baselines du Chapitre 3

Modèle	HF	BF	Mixte	Coût (CA)
<i>Animals-10</i>				
BL 1 (HF)	56,20%	35,16%	45,68%	470 400
BL 2 (BF)	59,07%	63,66%	61,36%	424 260
BL 3 (Mixte)	75,90%	65,03%	70,47%	894 660
Stratégie 1 (BF → HF)	78,84%	57,65%	68,25%	400 230
<i>Imagewoof</i>				
BL 1 (HF)	24,87%	19,04%	21,95%	179 800
BL 2 (BF)	43,73%	44,52%	44,12%	162 520
BL 3 (Mixte)	47,19%	47,24%	47,21%	342 320
Stratégie 1 (BF → HF)	53,68%	44,82%	49,25%	153 160

Comparaisons clés (Animals-10) :

- **Objectif principal HF atteint** : la stratégie 1 obtient **78,84% sur Test HF**, soit le **meilleur score HF de toutes les approches testées** (BL 3 = 75,90%, BL 2 = 59,07%, BL 1 = 56,20%).
- **Vs BL 3 (Mixte)** : tout en dépassant BL 3 sur HF (+2,94 points), la stratégie 1 réduit fortement le coût (894 660 → 400 230 CA, soit ≈ 55%).
- **Compromis BF accepté** : la précision BF reste inférieure à BL 2 et BL 3 (57,65% vs 63,66% et

65,03%), mais ce compromis est cohérent avec la priorité explicitement fixée sur la qualité HF.

Vérification sur Imagewoof

Sur Imagewoof, la Stratégie 1 atteint **53,68 %** en Test HF, soit **le meilleur score HF parmi les quatre modèles testés** (BL 3 = 47,19 %, BL 2 = 43,73 %, BL 1 = 24,87 %). L'avantage de la stratégie séquentielle est même plus marqué que sur Animals-10 :

- **Vs BL 3 (Mixte)** : +6,49 points HF pour un coût divisé par $\approx 2,2$ (153 160 vs 342 320 CA), confirmation directe de l'efficacité de la séparation BF $\rightarrow$ HF.
- **Rétention BF satisfaisante** : 44,82 % sur BF, comparable à BL 2 (44,52 %) ; l'écart HF/BF se resserre légèrement par rapport à Animals-10, ce qui suggère qu'un problème *fine-grained* à faible volume conserve une plus forte dépendance aux représentations basses fréquences.

La hiérarchie **Stratégie 1 > BL 3 > BL 2 > BL 1** est strictement identique sur les deux datasets, ce qui confirme que la dominance du transfer learning séquentiel n'est pas un artefact d'Animals-10 et constitue bien une propriété robuste de la méthode.

4.1.4 Analyse : ce qui a marché, ce qui a moins marché, et enseignements

- **Ce qui a bien marché (HF)** : la phase BF fournit une initialisation robuste, puis le fine-tuning HF spécialise efficacement le modèle ; le gain final en Test HF est net et place la stratégie 1 en tête.
- **Limite observée** : la spécialisation HF entraîne une perte partielle de robustesse BF (*domain forgetting*), ce qui était attendu dans une optimisation orientée domaine propre.
- **Signal de sur-apprentissage léger** : en fin de fine-tuning, la *train loss* continue de baisser tandis que la *val loss* HF se dégrade légèrement ; un *early stopping* piloté par Test/Val HF améliorerait encore la stabilité.
- **Leçon principale** : pour un objectif centré sur la performance HF, le transfer learning séquentiel offre le meilleur compromis observé entre précision cible et coût d'annotation.

Conclusion Étape 1 (objectif HF prioritaire) : la stratégie BF $\rightarrow$ HF est validée comme meilleure option actuelle pour maximiser la performance sur données propres. L'Étape 2 (co-training pondéré) visera désormais à conserver ce niveau HF tout en regagnant de la robustesse sur BF.

4.1.5 Améliorations possibles de la méthode (Stratégie 1)

Les résultats obtenus montrent que la stratégie séquentielle est déjà très compétitive sur HF, mais plusieurs leviers peuvent encore être optimisés.

- **Nombre d'époques BF et HF** : modifier la durée relative pré-entraînement/fine-tuning (par ex. 5/15, 8/12, 12/8) peut déplacer le compromis robustesse BF vs spécialisation HF. *Comment optimiser* : effectuer une recherche en grille sur le couple (E_{BF}, E_{HF}) et sélectionner la configuration

maximisant Test HF.

- **Learning rates par phase** : un LR trop élevé en fine-tuning peut détruire des représentations utiles, trop faible peut sous-adapter HF. *Comment optimiser* : tester plusieurs couples (lr_{BF}, lr_{HF}) avec schémas de décroissance (Cosine, StepLR, ReduceLROnPlateau).
- **Politique de gel/dégel des couches** : geler partiellement le backbone au début du fine-tuning peut limiter l'oubli BF tout en spécialisant la tête. *Comment optimiser* : comparer *full fine-tuning* vs *head-only* puis dégel progressif bloc par bloc.
- **Split HF train/validation** : la proportion 80/20 peut être remplacée (70/30, 85/15) pour stabiliser la sélection de modèle. *Comment optimiser* : valider sur plusieurs seeds et retenir la configuration la plus stable en variance de score HF.
- **Batch size** : influe sur la stabilité du gradient et la généralisation. *Comment optimiser* : explorer 32/64/96 (selon VRAM) en ajustant le LR selon une règle de mise à l'échelle.
- **Régularisation** : ajouter *weight decay*, *dropout* ou *label smoothing* peut réduire le sur-apprentissage en fin de phase HF. *Comment optimiser* : balayage de plages courtes (ex. $weight\ decay \in [10^{-5}, 10^{-3}]$, $label\ smoothing \in [0.0, 0.1]$).
- **Augmentations de données** : RandAugment, ColorJitter léger, RandomErasing peuvent améliorer la robustesse sans dégrader HF si bien dosés. *Comment optimiser* : ablations une augmentation à la fois puis combinaison des plus bénéfiques.
- **Early stopping et checkpointing** : arrêter sur la meilleure validation HF évite la dégradation tardive. *Comment optimiser* : utiliser une patience (3 à 5 époques) et conserver systématiquement le meilleur checkpoint HF.
- **Architecture backbone** : ResNet-18 est un bon compromis, mais ResNet-34 ou EfficientNet-B0 peuvent améliorer la représentation. *Comment optimiser* : comparer à budget de coût constant et même protocole d'évaluation.
- **Fonction de coût avancée** : Focal Loss ou marges additives peuvent mieux séparer des classes ambiguës en HF. *Comment optimiser* : tests contrôlés en remplaçant uniquement la loss, avec mêmes données et mêmes hyperparamètres de base.

4.2 Stratégie 2 – Co-training par Batch avec Pondération (Reweighting)

4.2.1 Protocole

Contrairement à la stratégie séquentielle $BF \rightarrow HF$, cette approche entraîne le modèle sur les deux domaines **au sein d'un même lot**. L'idée est de conserver la diversité BF tout en imposant une priorité d'optimisation sur les exemples HF, considérés comme plus fiables.

Deux mécanismes sont appliqués conjointement :

1. **Co-training à ratio fixe par batch** : chaque lot contient exactement 16 images HF et 48 images BF (batch size = 64), soit un ratio HF de 25%.
2. **Loss pondérée par domaine** : la perte individuelle est multipliée par un poids plus fort sur HF ($w_{HF} = 2.0$) et plus faible sur BF ($w_{BF} = 0.7$), avant agrégation.

Configuration expérimentale utilisée :

Paramètre	Valeur	Commentaire
Architecture	ResNet-18	Entraînée <i>from scratch</i> , classifieur 10 classes
Optimiseur	Adam	Même famille d'optimisation que les expériences précédentes
Batch size	64	Composition contrôlée HF/BF à chaque itération
Époques	20	Entraînement conjoint sur 117 batches par époque
Ratio batch	16 HF / 48 BF	Ratio HF fixé à 25% par <i>MixedBatchSampler</i>
Poids de loss	HF : 2.0 ; BF : 0.7	Accent mis sur les erreurs HF dans la mise à jour des gradients
Données utilisées	1 881 HF train + 21 213 BF train	Validation sur 471 HF ; test sur 2 614 images

4.2.2 Résultats quantitatifs obtenus

Les résultats finaux de l'Étape 2 sont résumés ci-dessous pour les deux datasets. Le critère prioritaire reste la précision sur **Test HF**.

Méthode	Test HF	Test BF	Test Mixte	Coût (CA)
<i>Animals-10</i>				
Stratégie 2 (Co-training pondéré)	70,12%	61,90%	66,01%	486 720
<i>Imagewoof</i>				
Stratégie 2 (Co-training pondéré)	39,20%	39,81%	39,50%	183 040

Lecture rapide des courbes d'entraînement (Animals-10)

La *train weighted loss* décroît de 2,148 (époque 1) à 0,365 (époque 20), avec une baisse plus rapide de la loss HF non pondérée (2,093 → 0,070) que de la loss BF (2,098 → 0,630), ce qui confirme l'effet attendu du reweighting. La validation HF progresse globalement de 30,57% à 69,85%, avec des oscillations intermédiaires (ex. époques 11, 14, 17) indiquant une optimisation plus instable qu'en stratégie séquentielle.

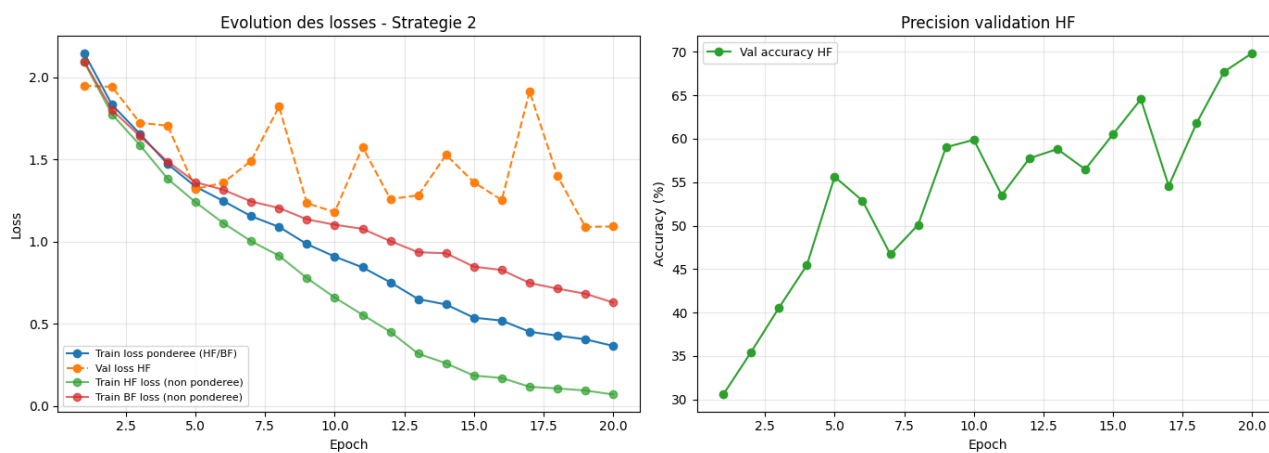


FIGURE 4.3 – Évolution des pertes et de la précision validation HF pour la Stratégie 2 (co-training pondéré HF/BF) sur **Animals-10**.

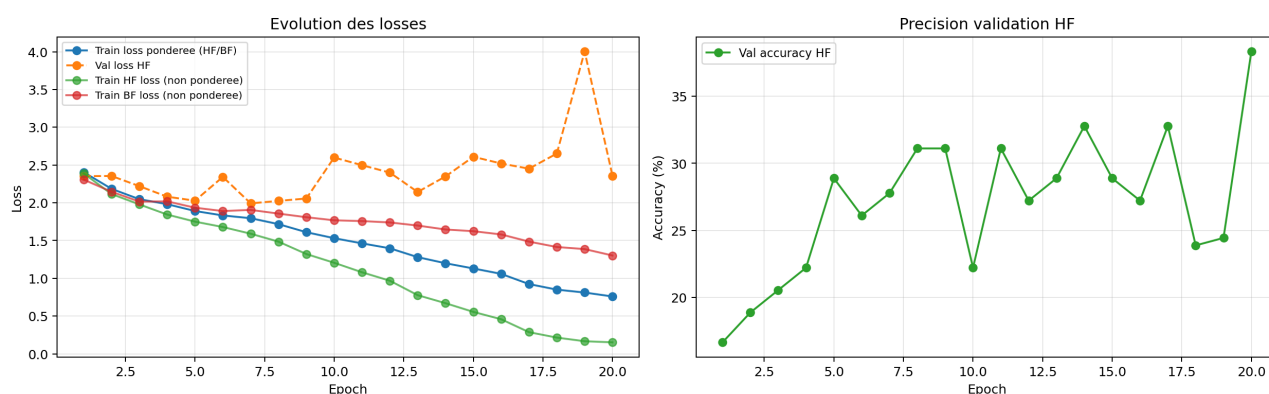
Strategie 2 (CoTrain pondere) - Imagewoof

FIGURE 4.4 – Évolution des pertes et de la précision validation HF pour la Stratégie 2 sur **Imagewoof**.

4.2.3 Comparaison avec les baselines et la Stratégie 1

Modèle	HF	BF	Mixte	Coût (CA)
<i>Animals-10</i>				
BL 1 (HF)	56,20%	35,16%	45,68%	470 400
BL 2 (BF)	59,07%	63,66%	61,36%	424 260
BL 3 (Mixte)	75,90%	65,03%	70,47%	894 660
Stratégie 1 (BF → HF)	78,84%	57,65%	68,25%	400 230
Stratégie 2 (Co-training pondéré)	70,12%	61,90%	66,01%	486 720
<i>Imagewoof</i>				
BL 1 (HF)	24,87%	19,04%	21,95%	179 800
BL 2 (BF)	43,73%	44,52%	44,12%	162 520
BL 3 (Mixte)	47,19%	47,24%	47,21%	342 320
Stratégie 1 (BF → HF)	53,68%	44,82%	49,25%	153 160
Stratégie 2 (Co-training pondéré)	39,20%	39,81%	39,50%	183 040

Comparaisons clés (Animals-10) :

- **Gain net vs BL 1 et BL 2 sur HF** : la stratégie 2 atteint 70,12% sur Test HF, soit +13,92 points vs BL 1 et +11,05 points vs BL 2.
- **Robustesse BF préservée** : avec 61,90% sur BF, la stratégie 2 reste proche de BL 2 (63,66%) et supérieure à la stratégie 1 (57,65%), ce qui traduit une meilleure conservation de la robustesse inter-domaines.
- **Compromis face à la stratégie 1** : la stratégie 2 perd 8,72 points sur HF par rapport à la stratégie 1 (70,12% vs 78,84%), mais améliore BF de +4,25 points.
- **Compromis coût** : le coût de la stratégie 2 (486 720 CA) reste bien inférieur à BL 3 (894 660 CA), mais supérieur à la stratégie 1 (400 230 CA).

Vérification sur Imagewoof

Sur Imagewoof, la Stratégie 2 atteint **39,20 %** sur Test HF — un résultat **nettement inférieur** aux baselines BL 2 (43,73 %) et BL 3 (47,19 %), et bien en deçà de la Stratégie 1 (53,68 %). Seule BL 1 (24,87 %) reste en dessous. Ce comportement diverge fortement de celui observé sur Animals-10 :

- **Comportement opposé à Animals-10** : sur Animals-10, la Stratégie 2 battait BL 1 et BL 2 de plus de +10 points HF. Sur Imagewoof, elle est devancée par BL 2 et BL 3 simples.
- **Robustesse inter-domaine néanmoins préservée** : malgré l'accuracy plus faible, l'écart HF/BF de la Stratégie 2 reste extrêmement resserré (39,20 vs 39,81), conforme à l'esprit de la méthode — meilleur équilibre HF/BF que toutes les autres approches sur ce dataset.
- **Hypothèse explicative** : sur Imagewoof, le co-training à ratio 16/48 par batch est mécaniquement dominé par les 48 échantillons BF, et le faible volume HF disponible (719 images train HF contre 1 881 sur Animals-10) ne suffit pas à ce que le *reweighting* HF spécialise efficacement les features fine-grained. La Stratégie 1, qui sépare strictement les phases BF puis HF, contourne ce problème.

Cette divergence est précieuse pour l'analyse : elle indique que les performances de la Stratégie 2 sont **sensibles au volume HF disponible** et au degré de granularité du problème. Une étude d'ablation du ratio HF/BF et des poids de loss serait nécessaire pour la rendre transposable sur des datasets fine-grained à faible volume.

4.2.4 Analyse : ce qui a marché, ce qui a moins marché, et enseignements

- **Ce qui a bien marché (équilibre des domaines)** : le mélange systématique HF/BF dans chaque batch limite l'oubli de domaine et maintient un niveau BF solide tout en dépassant nettement les baselines mono-domaine sur HF.
- **Effet du reweighting confirmé** : la décroissance plus rapide de la loss HF non pondérée montre que le modèle a effectivement priorisé les erreurs HF pendant l'optimisation.
- **Limite principale (objectif HF prioritaire)** : malgré la pondération HF, la stratégie 2 ne retrouve pas le niveau HF de la stratégie 1, suggérant qu'une spécialisation finale explicite sur HF reste plus efficace lorsqu'on vise la performance maximale sur domaine propre.
- **Leçon principale** : le co-training pondéré constitue une stratégie de compromis robuste HF/BF, mais pas l'option optimale si la cible unique est la précision HF maximale.

Conclusion Étape 2 (objectif HF prioritaire) : la stratégie de co-training pondéré valide sa pertinence pour stabiliser le compromis inter-domaines, mais la stratégie 1 reste meilleure sur la métrique cible Test HF. Les étapes suivantes devront chercher à conserver ce gain HF tout en maintenant la robustesse BF observée ici.

4.2.5 Améliorations possibles de la méthode (Stratégie 2)

La stratégie de co-training pondéré offre une bonne robustesse inter-domaines ; son optimisation repose surtout sur le réglage fin du mélange de données et des poids de perte.

- **Ratio HF/BF dans le batch** : le choix 16/48 (25% HF) conditionne directement le compromis HF-BF. *Comment optimiser* : tester plusieurs ratios (20/80, 30/70, 40/60) et tracer la frontière de Pareto entre Test HF et Test BF.
- **Poids de loss w_{HF} et w_{BF}** : ce sont les hyperparamètres centraux de la méthode. *Comment optimiser* : recherche en grille ou bayésienne sur (w_{HF}, w_{BF}) (ex. $w_{HF} \in [1.0, 3.0]$, $w_{BF} \in [0.4, 1.0]$), en fixant d'abord le ratio batch.
- **Planning dynamique des poids** : garder des poids constants peut être sous-optimal selon la phase d'apprentissage. *Comment optimiser* : démarrer avec un poids HF modéré puis l'augmenter progressivement (curriculum de pondération).
- **Nombre d'époques** : des oscillations de validation suggèrent qu'un arrêt plus précoce peut être préférable à 20 époques fixes. *Comment optimiser* : early stopping sur validation HF avec patience et sauvegarde du meilleur modèle.
- **Learning rate et scheduler** : un LR fixe peut accentuer l'instabilité tardive. *Comment optimiser* : comparer LR fixes vs Cosine Annealing / OneCycleLR pour lisser la convergence.
- **Composition des batches (stochastique contrôlée)** : la variabilité intra-batch peut influencer la stabilité. *Comment optimiser* : conserver le ratio global mais imposer une diversité de classes par domaine (batch balancing par classe).
- **Fonction de loss alternative** : Balanced Softmax, focal reweighting, ou class-balanced loss peuvent mieux traiter classes rares et bruitées. *Comment optimiser* : protocole d'ablation en ne changeant qu'une composante de loss à la fois.
- **Qualité de la dégradation BF** : le niveau de bruit/sous-échantillonnage détermine l'écart de domaine. *Comment optimiser* : calibrer plusieurs intensités de dégradation puis choisir celle qui maximise la généralisation HF sans effondrer BF.
- **Régularisation et normalisation** : weight decay, EMA des poids, mixup/cutmix peuvent améliorer la robustesse de l'entraînement mixte. *Comment optimiser* : introduire les techniques progressivement et évaluer le gain marginal.
- **Validation multi-critères** : optimiser uniquement HF peut dégrader excessivement BF. *Comment optimiser* : utiliser un score composé (ex. $S = \alpha Acc_{HF} + (1 - \alpha) Acc_{BF}$) avec α élevé, cohérent avec la priorité HF.

4.3 Stratégie 3 – Teacher-Student inspirée de Noisy Student

4.3.1 Protocole

Cette stratégie adapte le schéma *Teacher-Student* de Xie et al. (2020) à notre contexte multi-fidélité :

1. **Teacher HF** : un ResNet-18 est entraîné uniquement sur HF (10 époques), puis figé en mode `eval()`.
2. **Pseudo-labeling BF** : le Teacher produit des *soft pseudo-labels* (probabilités Softmax) pour l'ensemble BF d'entraînement.
3. **Student noised** : un nouveau ResNet-18 est entraîné *from scratch* avec augmentation agressive (RandAugment, ColorJitter, RandomErasing, flips), sur un mélange :
 - HF avec labels réels (`CrossEntropyLoss`),
 - BF avec pseudo-labels mous (*Soft Cross-Entropy*).

La perte globale est définie par :

$$\mathcal{L}_{tot} = \mathcal{L}_{HF} + \alpha \cdot \mathcal{L}_{BF}, \quad \alpha = 1.0$$

Configuration expérimentale utilisée :

Paramètre	Valeur	Commentaire
Architecture Teacher/Student	ResNet-18 / ResNet-18	Student avec dropout (p=0,3) sur la tête
Entraînement Teacher	10 époques HF	HF train = 1 881 ; HF val = 471
Pseudo-labeling BF	Softmax (température 1,0)	21 213 images BF pseudo-labellisées
Entraînement Student	20 époques HF+BF	Loss mixte hard HF + soft BF
Pondération BF	$\alpha = 1,0$	Même poids global HF/BF dans la loss
Augmentations Student	Agressives	RandAugment, ColorJitter, RandomErasing, flips

4.3.2 Résultats quantitatifs obtenus

Les résultats finaux de l'Étape 3 sont résumés ci-dessous. Le critère prioritaire reste la précision sur **Test HF**.

Méthode	Test HF	Test BF	Test Mixte	Coût (CA)
Stratégie 3 (Noisy Student adapté)	52,72%	28,46%	40,59%	988 560

Lecture rapide des courbes d'entraînement

Le Teacher progresse en validation HF jusqu'à environ 45,65% (époque 9), puis termine à 43,10% (époque 10), indiquant une qualité de pseudo-labels limitée. Le Student réduit sa loss d'entraînement (4,46 → 3,13) et atteint 50,11% en validation HF (époque 19), mais la généralisation finale reste insuffisante sur les trois tests.

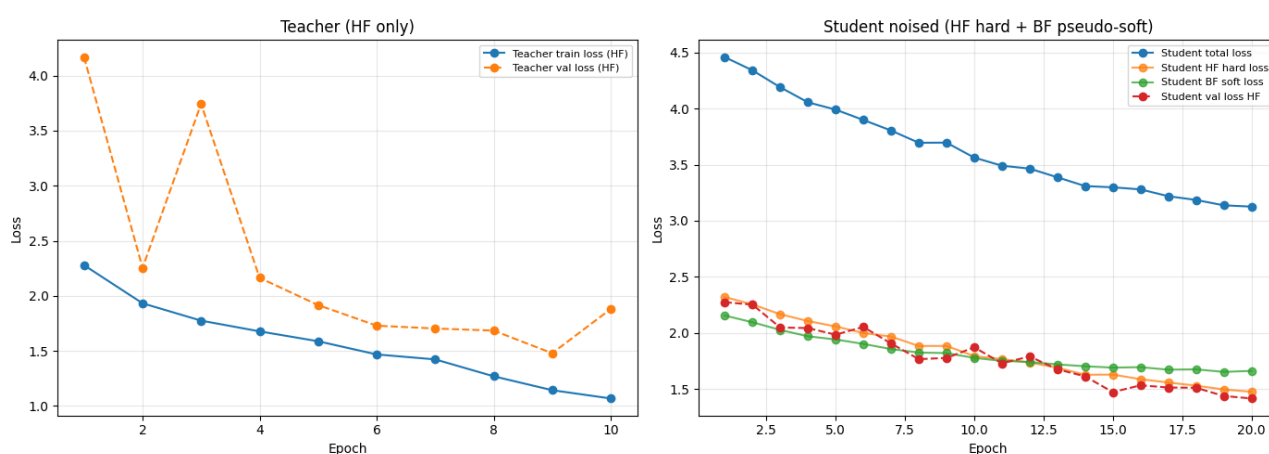


FIGURE 4.5 – Évolution des pertes Teacher/Student pour la Stratégie 3 (Noisy Student adapté au cadre multi-fidélité).

4.3.3 Comparaison avec les autres stratégies

Modèle	HF	BF	Mixte	Coût (CA)
BL 1 (HF)	56,20%	35,16%	45,68%	470 400
BL 2 (BF)	59,07%	63,66%	61,36%	424 260
BL 3 (Mixte)	75,90%	65,03%	70,47%	894 660
Stratégie 1 (BF → HF)	78,84%	57,65%	68,25%	400 230
Stratégie 2 (Co-training pondéré)	70,12%	61,90%	66,01%	486 720
Stratégie 3 (Noisy Student adapté)	52,72%	28,46%	40,59%	988 560

Comparaisons clés :

- **Objectif HF non atteint** : avec 52,72% sur HF, la stratégie 3 est en dessous de BL 1 (56,20%) et très loin des stratégies 1 et 2.
- **Effondrement BF** : la précision BF chute à 28,46%, soit la plus faible de toutes les approches testées.
- **Coût défavorable** : le coût total (988 560 CA) devient le plus élevé du panel, sans gain de performance.
- **Bilan global** : la stratégie 3 est dominée simultanément en précision et en coût par les stratégies précédentes.

4.3.4 Analyse : pourquoi la stratégie 3 échoue dans ce contexte

- **Teacher insuffisamment performant** : un pseudo-labeling n'est utile que si le Teacher est déjà solide. Ici, sa validation HF reste modeste, ce qui injecte du bruit sémantique dans BF.
- **Accumulation de bruit** : BF est déjà un domaine dégradé ; lui ajouter des pseudo-labels imparfaits puis des augmentations agressives sur le Student peut amplifier l'erreur au lieu de la corriger.
- **Désalignement de domaine** : dans cette configuration, les pseudo-labels BF ne compensent pas le décalage HF/BF, et le Student n'apprend pas de représentations transférables vers le test HF.
- **Poids BF potentiellement trop élevé** : avec $\alpha = 1.0$, la composante BF peut dominer l'optimisation malgré la priorité HF du projet.

Conclusion Étape 3 (objectif HF prioritaire) : l'adaptation directe de Noisy Student n'est pas validée dans notre cadre multi-fidélité actuel. Les performances chutent fortement alors que le coût augmente. Les prochaines variantes devront d'abord renforcer la qualité du Teacher et réduire l'influence des pseudo-labels BF (filtrage par confiance, pondération BF plus faible, augmentations moins agressives, ou Teacher plus robuste).

4.3.5 Améliorations possibles de la méthode (Stratégie 3)

Pour rendre cette approche exploitable, plusieurs paramètres critiques doivent être ré-optimisés :

- **Qualité initiale du Teacher** : c'est le facteur principal. *Comment optimiser* : entraîner plus longtemps, ajuster LR/scheduler, et sélectionner le meilleur checkpoint val HF avant pseudo-labeling.
- **Seuil de confiance pseudo-labels** : utiliser tous les pseudo-labels propage les erreurs. *Comment optimiser* : ne conserver que les échantillons BF au-dessus d'un seuil de confiance (ex. max prob $> 0,7/0,8$).
- **Température de pseudo-labeling** : la température contrôle la netteté des distributions. *Comment optimiser* : balayer $T \in [0, 7, 2, 0]$ et mesurer l'impact sur HF final.

- **Pondération BF (α)** : $\alpha = 1$ peut sur-pondérer BF bruité. *Comment optimiser* : tester des valeurs plus faibles (0,2 ; 0,4 ; 0,6 ; 0,8) avec priorité sur Test HF.
- **Intensité du bruit Student** : augmentation trop agressive peut détruire les signaux utiles. *Comment optimiser* : réduire progressivement RandAugment/ColorJitter/RandomErasing et faire des ablations.
- **Mix HF/BF par batch** : un mélange non contrôlé peut défavoriser HF. *Comment optimiser* : imposer un ratio HF minimal par batch (sampler dédié) pour ancrer l'optimisation sur les labels fiables.
- **Architecture Teacher/Student** : un Teacher plus capacitaire peut améliorer la qualité des pseudo-labels. *Comment optimiser* : Teacher ResNet-34/EfficientNet-B0, Student plus léger ou équivalent, à protocole constant.
- **Fonction de perte BF** : la Soft CE simple peut être insuffisante. *Comment optimiser* : tester KLDiv, entropie régularisée, ou pondération par confiance échantillon.
- **Curriculum d'apprentissage** : lancer d'emblée HF+BF pseudo peut être instable. *Comment optimiser* : warm-up Student sur HF seul, puis injection progressive des données BF pseudo-labellisées.

Bibliographie

- [1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. *Deep Residual Learning for Image Recognition*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- [2] Diederik P. Kingma, Jimmy Ba. *Adam : A Method for Stochastic Optimization*. International Conference on Learning Representations (ICLR), 2015.
- [3] Ian Goodfellow, Yoshua Bengio, Aaron Courville. *Deep Learning*. MIT Press, 2016. Disponible en ligne : <https://www.deeplearningbook.org>
- [4] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, Li Fei-Fei. *Image-Net Large Scale Visual Recognition Challenge*. International Journal of Computer Vision (IJCV), 123(3) :211–252, 2015.
- [5] Dan Hendrycks, Thomas Dietterich. *Benchmarking Neural Network Robustness to Common Corruptions and Perturbations*. International Conference on Learning Representations (ICLR), 2019.
- [6] Marc Peter Deisenroth, Jun Wei Ng. *Distributed Gaussian Processes*. International Conference on Machine Learning (ICML), 2015.
- [7] Marc C. Kennedy, Anthony O'Hagan. *Predicting the Output from a Complex Computer Code When Fast Approximations Are Available*. Biometrika, 87(1) :1–13, 2000.
- [8] Mengye Ren, Wenyuan Zeng, Bin Yang, Raquel Urtasun. *Learning to Reweight Examples for Robust Deep Learning*. International Conference on Machine Learning (ICML), 2018.
- [9] Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor Tsang, Masashi Sugiyama. *Co-teaching : Robust Training of Deep Neural Networks with Extremely Noisy Labels*. Advances in Neural Information Processing Systems (NeurIPS), 2018.
- [10] Qizhe Xie, Minh-Thang Luong, Eduard Hovy, Quoc V. Le. *Self-training with Noisy Student improves ImageNet classification*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020.